

Phi-4 경량화 진행 상황

2026-06-24

CONTENTS

Phi-4 경량화 진행 상황 (2026-06-24)	
1. 프루닝 기법 — 적용 결과	
1.1 적용 성공 (4종)	
1.2 적용 실패 (2종) + 이유	
2. 무엇을 얼마나 프루닝했나 + 왜	
3. 기본 모델 대비 비교 (KMMLU, limit=20, 회복 전)	
4. Distillation 회복 결과 (depth-pruned 모델 대상)	
평가 지표 & 방법 (도구)	
5. 결론	
6. Reference	
모델	
프루닝 기법 (논문 / 레포)	
학습 데이터 (Distillation — teacher 응답 생성용 프롬프트 풀)	
평가 데이터 / 벤치마크	
도구	

Phi-4 경량화 진행 상황 (2026-06-24)

기본 모델: Phi-4

비교 모델: Phi-4-mini-instruct Q4_K_M을 한국어 데이터셋으로 비교

GPU: RTX PRO 6000 Blackwell 96GB

1. 프루닝 기법 — 적용 결과

1.1 적용 성공 (4종)

기법	자르는 축	중요도 기준	구현
ShortGPT	depth (레이어 통째)	Block Influence (레이어 입·출력 코사인 유사도)	직접 구현 (순수 PyTorch)
Minitron-식 width	width — MLP intermediate 뉴런	활성값(activation)	직접 구현
magnitude width	width — MLP intermediate 뉴런	weight IWI (L2)	직접 구현
SliceGPT	width — hidden 차원 (d_model) PCA	residual stream PCA	공식 lib (microsoft/TransformerCompression)

1·2·3 (직접 구현) — 가중치 행렬을 손으로 잘라낸다

- 외부 라이브러리 없이 가중치 행렬에서 덜 중요한 부분을 직접 잘라낸다(수동 슬라이싱).
- 핵심은 “짜깁 레이어를 같이 잘라야 한다”는 점. 예를 들어 MLP에서 `gate_up_proj`의 출력 뉴런과 `down_proj`의 입력 뉴런은 같은 중간 뉴런을 공유한다. 그래서 남길 뉴런 목록을 하나 정해, 두 행렬을 그 같은 목록으로 함께 잘라 차원을 실제로 줄인다.
- 0으로 덮는 마스킹이 아니라 텐서를 진짜 작게 만들기 때문에, llama.cpp(dense)·GGUF에서 그대로 메모리가 줄어든다.

4 (SliceGPT) — 모델의 “폭” 자체를 줄인다

- 유일하게 외부 공식 라이브러리 사용(Phi-4는 공식 미지원이라 Phi-3용 설정을 이름만 맞춰 patch).
- 자르는 축이 1·2·3과 다르다. 1·2·3은 “MLP 중간 뉴런”이나 “레이어”를 줄이지만, SliceGPT는 모델 전체를 관통하는 hidden 차원(d_model, 5120) 자체를 줄인다(= 배관 굵기를 줄이는 셈).

- 방식: 트랜스포머의 수학적 성질(출력을 바꾸지 않는 회전을 끼워넣을 수 있음)을 이용해 가중치를 **회전**시킨 뒤, **PCA**로 정보가 거의 없는 축을 찾아 잘라낸다.

1.2 적용 실패 (2종) + 이유

라이브러리	결과	이유
Torch-Pruning (VainF)	Phi-4에서 그래프 자동 추적 무한루프	아래 상세
LLM-Pruner	Phi 미지원	torch-pruning 기반. 공식 지원: Llama-2/3, BLOOM, Vicuna, Baichuan, TinyLlama — Phi는 없음

Torch-Pruning이 Phi-4에서 안 되는 이유:

- torch-pruning은 채널 의존성 파악을 위해 **실제 forward**를 한 번 돌려 그래프를 자동 추적한다.
- transformers 5.x의 **Phi-4 동적 attention forward**(`qkv_proj` → `split` → `view/transpose` → `rotary` → **DynamicCache** 갱신 → `repeat_kv` 확장 → `matmul` → `o_proj`)에서 그 추적이 폭주 → **무한루프**. (CPU·GPU 둘 다 직접 확인: 90초/38분 timeout)
- **Phi-4만 안 되고 Qwen 등은 되는 핵심 차이 = fused projection**. Phi-3/Phi-4는 QKV·gate/up이 한 **행렬로 fused**(내부 `split/chunk`로 분기)되어 추적기가 의존성을 모호하게 처리. Qwen2/Llama는 q/k/v·gate/up이 **분리**돼 있어 추적이 깔끔. DynamicCache·rotary·GQA 자체는 Qwen에도 있으므로 그것만으론 원인이 아님.
- 추적이 됐더라도 fused qkv/gate_up·GQA 결합을 정확히 자르기 어려움(부차적 문제).

2. 무엇을 얼마나 프루닝했나 + 왜

기법	프루닝 대상	양	파라미터	근거
ShortGPT (depth)	Transformer 레이어	40층 → 28층 (12층 제거, ~28%)	14.66B → ~10.6B	Block Influence로 잉여 레이어 (중간후반 2737) 식별. 레이어 0· 끝부분은 중요 → 보존 (ShortGPT 논문 Phi-4 재현)
width (Minitron/magnitude)	MLP intermediate 뉴런	17920 → ~12544 (30%)	→ ~11.4B (~22.5%)	MLP는 결합구조 단순(gate/up 출력 = down 입력)해 안전. 활성 값=한국어에 실제 반응하는 뉴런 보존
SliceGPT	hidden 차원 (d_model)	5120 → 3584 (30% sparsity)	14.66B → 11.48B (~21.7%)	residual stream을 PCA로 회전 후 저정보 축 슬라이스

왜 이렇게 했나 (전략):

- **structured**(차원 실제 축소)만 메모리 실감소 → llama.cpp dense/GGUF 직결. masking(Wanda 등)·2:4 sparse는 sparse 런타임 한정이라 제외.
- 두 축(**depth vs width**) + 중요도 기준(**활성값 vs magnitude**)을 통제 비교 → “Phi-4엔 무엇이 유리한가” 정량 결론 도출.
- 감축률을 width 계열은 ~30%로 맞춰 공정 비교.

3. 기본 모델 대비 비교 (KMMLU, limit=20, 회복 전)

모델	축	KMMLU	원본대비
원본 Phi-4 (14.66B)	—	34.11%	100%
ShortGPT depth (~10.6B)	depth	31.89%	~94%
Minitron width (~11.4B)	width(활성값)	18.33%	~54%
SliceGPT (11.48B)	width(hidden PCA)	8.72%	~26%
magnitude width (~11.4B)	width(IWI)	7.33%	~21%
Phi-4-mini Q4_K_M (3.8B)	기성 소형	30.33%	—

해석:

- **depth(ShortGPT) 압도적 승리** — 더 줄이면서 정확도 거의 유지. **full(35,030문항) 재측정 시 34.92%** 로 원본 소폭 상회(=사실상 무손실).
- width 계열은 회복 전 큰 손실. **활성값(18.33%) » magnitude(7.33%)** — “한국어에 반응하는 뉴런”을 살리는 게 결정적.

측정 조건: limit=20(45과목×20=900문항)이라 $\pm 2\sim 3\%$ p 노이즈

4. Distillation 회복 결과 (depth-pruned 모델 대상)

가장 우수했던 **depth-pruned(10.6B)** student를 teacher(`microsoft/phi-4`)로 **sequence-level KD**(teacher가 한국어 instruction에 답변 생성 → student LoRA SFT). full KMMLU(35,030) 기준:

단계	KMMLU	train_loss
pruned_depth (distill 전)	34.92%	—
distill (teacher 답변 24,958)	42.38%	0.688
원본 Phi-4 (참고)	34.11%	—
Phi-4-mini Q4 baseline	30.33%	—

학습 데이터 형태 (**sequence-level distillation**):

데이터를 직접 라벨링하지 않고, teacher 모델이 “모범 답안”을 만들어 student가 따라 배우게 한다. 3단계로 만든다:

1. **프롬프트 풀 수집** — 한국어 instruction(질문/지시) 약 24,958개 (KoCommercial-Dataset, kullm-v2, 합성 Ko-IFEval-style).
2. **teacher가 답변 생성** — phi-4(14B)가 각 프롬프트에 답을 생성 → (프롬프트, teacher 답변) 쌍 24,958개. (vLLM으로 고속 생성)
3. **student가 모방 학습** — pruned student를 이 쌍들로 LoRA SFT. teacher의 답을 출력하도록 학습.

포맷이 핵심 — 내용은 Q&A이지만 “대화(chat) 형식”으로 감싸 학습한다 (멀티턴 대화가 아니라 user→assistant 단일 턴):

```
<|im_start|>user
{한국어 instruction}<|im_end|>
<|im_start|>assistant
{teacher 답변}<|im_end|>
```

- 내용은 Q&A·지시수행형, 형식은 위 chat 템플릿(<|im_start|> 등 특수 토큰)으로 감쌌.
- 그래서 student는 ”chat 템플릿으로 감싼 입력”에만 익숙해진다 → **chat-template 의존적**.
- 평가 때 raw 프롬프트(템플릿 없이 instruction 텍스트만)를 주면 student가 곧장 종료 토큰을 뱉어 **빈 출력으로 붕괴**한다. 반드시 chat 템플릿을 적용해야 정상 동작 — 실제로 Ko-IFEval 첫 측정에서 이 현상으로 점수가 무너졌고, chat 엔드포인트로 바꾸자 정상화됨.

GGUF Q4_K_M 변환 후 baseline 비교:

지표	distill Q4 (우리)	Phi-4-mini Q4 (baseline)	판정
KMMLU (정확도, 높을수록↑)	42.38%	30.33%	우위 +12pt
PPL (ko_corpus, 낮을수록↑)	5.31	12.64	우위
Ko-IFEval prompt-strict (↑)	26.6%	32.5%	열세
Ko-IFEval inst-loose (↑)	38.7%	42.2%	열세
파일 크기	6.2GB	2.49GB	baseline

해석:

- **depth** 프루닝은 KMMLU 손실 거의 없음(34.11→34.92), 그 위에 **distillation**이 42.38%까지 끌어올림 = 원본·baseline 모두 상회.
- **PPL**도 절반 이하(5.31 vs 12.64)로 우수. 단 phi-4(vocab 10만)≠phi-4-mini(vocab 20만) 토큰나이저라 per-token 절대비교엔 caveat.
- **Ko-IFEval(지시준수)만 baseline에 열세** — student가 1 epoch LoRA SFT만 받아 full instruct-tuning된 phi-4-mini를 못 따라감. (단, distilled는 **chat-template** 의존적 — raw 프롬프트엔 빈 출력 붕괴하므로 `--apply_chat_template` +chat 엔드포인트로만 정상 평가됨)

평가 지표 & 방법 (도구)

지표	무엇을 보나	측정 방식	도구 (라이브러리?)
KMMLU	한국어 지식·추론 (4지선 다 정확도↑)	각 보기의 로그우도(loglikelihood) 비교→argmax 정답률. 생성 안 함	lm-evaluation-harness (라이브러리). HF모델= <code>hf</code> , SliceGPT=slicegpt로더 +HFLM, GGUF=llama.cpp 서버
PPL (퍼플렉시티)	언어모델링 능력 (다음 토큰 예측, 낮을수록↑)	<code>exp(토큰당 평균 loss)</code> . ko_corpus를 2048토큰 창으로 슬라이딩, 실제 다음 토큰 확률의 로그 평균	llama.cpp llama-perplexity (전용 바이너리, lm-eval 아님). GGUF 대상
Ko-IFEval	지시준수 (생성형↑)	아래 상세	lm-evaluation-harness <code>ifeval_ko</code> (allganize) + llama.cpp 서버 chat 엔드포인트 + <code>--apply_chat_template</code>

Ko-IFEval 평가 과정:

- 프롬프트에 **자동 검증 가능한 지시**가 들어있다 (예: “300단어 이상”, “심표 쓰지 마라”, “마크다운 섹션 3개 이상”, “[placeholder] 12개 포함”).
- 모델이 그 프롬프트에 **답을 생성**한다.
- 규칙 기반 검증기(코드)**가 생성된 답이 각 지시를 만족하는지 판정한다 — **LLM judge가 아니라** 단어 수·세기·심표 유무·정규식 같은 결정적 검사.
- 4개 점수: **prompt-level**(프롬프트의 모든 지시 만족?) / **instruction-level**(개별 지시 단위 충족 비율), 각각 **strict**(엄격) / **loose**(마크다운 제거 등 사소한 정규화 후 판정).

도구 한눈에:

- 프루닝 1·2·3 = 라이브러리 없이 직접 구현 / SliceGPT = microsoft/TransformerCompression
- distillation 답변 생성 = **vLLM**(고속) / 학습 = **HuggingFace PEFT**(LoRA SFT)
- GGUF 변환·PPL·서빙 = **llama.cpp** / 정확도·지시준수 평가 = **lm-evaluation-harness**

5. 결론

결론:

- Phi-4엔 **depth** 프루닝(**ShortGPT**)이 최선 — width(Minitron/magnitude/SliceGPT)는 회복 전 손실이 큼.

2. 중요도 기준이 결정적 — 활성값 $\gg$ magnitude.
 3. **depth-pruned + distillation** = 원본/baseline을 KMMLU·PPL에서 상회, Ko-IFEval만 열세.
 4. 기성 자동 프루닝 라이브러리(torch-pruning/LLM-Pruner)는 Phi-4에 부적합 → 수동 슬라이싱이 견고.
-

6. Reference

모델

- Phi-4 (기본 모델): https://huggingface.co/microsoft/phi-4SMARTPANTS_RESERVED_771SMARTPANTS_RESERVED_772
- Phi-4-mini-instruct (비교 모델): https://huggingface.co/microsoft/Phi-4-mini-instructSMARTPANTS_RESERVED_775SMARTPANTS_RESERVED_776

프루닝 기법 (논문 / 레포)

- [ShortGPT \(depth, Block Influence\): https://arxiv.org/abs/2403.03853SMARTPANTS_RESERVED_783SMARTPANTS_RESERVED_784](https://arxiv.org/abs/2403.03853)
- [Minitron \(width, 활성값 중요도 + distillation\): https://arxiv.org/abs/2407.14679SMARTPANTS_RESERVED_787SMARTPANTS_RESERVED_788](https://arxiv.org/abs/2407.14679)
- [SliceGPT \(hidden PCA slicing\): 논문 https://arxiv.org/abs/2401.15024SMARTPANTS_RESERVED_791 · 레포 https://github.com/microsoft/TransformerCompressionSMARTPANTS_RESERVED_793SMARTPANTS_RESERVED_794](https://arxiv.org/abs/2401.15024)
- [Torch-Pruning \(적용 실패\): https://github.com/VainF/Torch-PruningSMARTPANTS_RESERVED_797SMARTPANTS_RESERVED_798](https://github.com/VainF/Torch-Pruning)
- [LLM-Pruner \(Phi 미지원\): https://github.com/horseee/LLM-PrunerSMARTPANTS_RESERVED_801SMARTPANTS_RESERVED_802](https://github.com/horseee/LLM-Pruner)

학습 데이터 (DISTILLATION — TEACHER 응답 생성용 프롬프트 풀)

- [MarkrAI/KoCommercial-Dataset: https://huggingface.co/datasets/MarkrAI/KoCommercial-DatasetSMARTPANTS_RESERVED_809SMARTPANTS_RESERVED_810](https://huggingface.co/datasets/MarkrAI/KoCommercial-Dataset)
- [nlpai-lab/kullm-v2: https://huggingface.co/datasets/nlpai-lab/kullm-v2SMARTPANTS_RESERVED_813SMARTPANTS_RESERVED_814](https://huggingface.co/datasets/nlpai-lab/kullm-v2)
- [Synthetic Ko-IFEval-style prompts \(직접 생성, 외부 URL 없음\)](#)

평가 데이터 / 벤치마크

- [KMMLU \(한국어 MMLU\): https://huggingface.co/datasets/HAERAE-HUB/KMMLUSMARTPANTS_RESERVED_823SMARTPANTS_RESERVED_824](https://huggingface.co/datasets/HAERAE-HUB/KMMLUSMARTPANTS_RESERVED_823SMARTPANTS_RESERVED_824)

- [Ko-IFEval \(한국어 지시준수, allganize\): https://huggingface.co/datasets/allganize/IFEval-KoSMARTPANTS_PRESERVED_827SMARTPANTS_PRESERVED_828](https://huggingface.co/datasets/allganize/IFEval-KoSMARTPANTS_PRESERVED_827SMARTPANTS_PRESERVED_828)
- [한국어 PPL corpus: 내부 ko_corpus.txt](#)

도구

- [lm-evaluation-harness \(KMMLU/Ko-IFEval\): https://github.com/EleutherAI/lm-evaluation-harnessSMARTPANTS_PRESERVED_837SMARTPANTS_PRESERVED_838](https://github.com/EleutherAI/lm-evaluation-harnessSMARTPANTS_PRESERVED_837SMARTPANTS_PRESERVED_838)